

Ada-MK: Adaptive MegaKernel Optimization via Automated DAG-based Search for LLM Inference

Wenxin Dong, Mingqing Hu, Guanghui Yu, Qiang Fu, Peng Xu, Hui Xu, Yue Xing, Xuewu Jiao[†],
Shuanglong Li, Lin Liu
Baidu Inc.
Beijing, China

Abstract

When large language models (LLMs) serve real-time inference in commercial online advertising systems, end-to-end latency must be strictly bounded to the millisecond range. Yet every token generated during the decode phase triggers thousands of kernel launches, and kernel launch overhead alone can account for 14.6% of end-to-end inference time. MegaKernel eliminates launch overhead and inter-operator HBM round-trips by fusing multiple operators into a single persistent kernel. However, existing MegaKernel implementations face a fundamental tension between portability and efficiency on resource-constrained GPUs such as NVIDIA Ada: hand-tuned solutions are tightly coupled to specific architectures and lack portability, while auto-compiled approaches introduce runtime dynamic scheduling whose branch penalties are unacceptable in latency-critical settings.

We observe that under a fixed deployment configuration, the optimal execution path of a MegaKernel is uniquely determined, and runtime dynamic decision-making can be entirely hoisted to compile time. Building on this insight, we propose **Ada-MK**: (1) a three-dimensional shared-memory constraint model combined with K-dimension splitting that reduces peak shared memory usage by 50%; (2) MLIR-based fine-grained DAG offline search that solidifies the optimal execution path, completely eliminating runtime branching; and (3) a heterogeneous hybrid inference engine that embeds MegaKernel as a plugin into TensorRT-LLM, combining high-throughput Prefill with low-latency Decode. On an NVIDIA L20, Ada-MK improves single-batch throughput by up to 23.6% over vanilla TensorRT-LLM and 50.2% over vLLM, achieving positive gains across all tested scenarios—the first industrial deployment of MegaKernel in a commercial online advertising system.

Keywords

MegaKernel, DAG-based Optimization, Adaptive Optimization, LLM Inference, Shared Memory Reuse, Commercial Advertising

1 Introduction

Large language models (LLMs) are increasingly deployed in search, recommendation, and conversational applications, with parameter counts growing from billions to hundreds of billions. Inference latency has thus become a key bottleneck for industrial adoption, especially in latency-sensitive online services where efficient inference under limited hardware resources is critical. For extremely latency-sensitive scenarios, frequent global memory accesses and kernel launch overhead on GPUs have become the primary bottlenecks for end-to-end performance [13, 21]. Profiling Qwen2.5-1.5B with Nsight Systems under TensorRT-LLM reveals that kernel launch overhead accounts for approximately 14.6% of end-to-end

inference time (1,655,550 launches consuming ~ 3.3 s). In conventional pipelines, adjacent kernels must exchange intermediate results through HBM; MegaKernel instead leverages shared memory and registers to achieve seamless operator chaining and deep parallelism, fundamentally eliminating operator switching overhead and HBM round-trip latency. MegaKernel achieves single-launch persistent computation through Persistent Kernels, constructs producer-consumer pipelines via Warp Specialization, and overlaps computation with memory access through TMA and asynchronous I/O [9].

Our target deployment is a commercial online advertising system running on NVIDIA Ada (L20) GPUs [18]. Deploying MegaKernel on Ada faces severe dual constraints. First, the business demands deterministic end-to-end latency strictly within 1–5 ms. Second, the Ada architecture natively lacks TMA hardware support, requiring PTX assembly and hand-crafted software pipelines to emulate asynchronous data movement; its on-chip shared memory is only half that of the H100 (128 KB vs. 227 KB) [19], severely compressing the optimization space for pipeline stages and tile sizes. In contrast, Hopper/Blackwell architectures provide TMA hardware support and larger shared memory (227 KB), making MegaKernel deployment far more straightforward [9, 19]. In practice, the shared memory limitation on Ada reduces the achievable pipeline stages from the theoretical optimum of 4 to only 2, incurring a pipeline duty-cycle loss exceeding 30%.

Existing solutions fail to meet these requirements. Stanford’s MegaKernel [9] delivers strong performance, but its codebase is deeply tied to Hopper/Blackwell-specific assembly optimizations and supports only a few model architectures (e.g., Llama-1B), lacking support for Qwen and other widely-used models [23, 24]; it also lacks support for long-sequence and large-batch Prefill phases. Mirage MPK [1, 27] improves usability through auto-tuning, but its Managed Pointer mechanism introduces runtime if-else branching based on shared memory page states, degrading instruction issue efficiency in ultra-low-latency scenarios and falling short of hand-tuned operator performance.

To address these challenges, we propose Ada-MK with the following core contributions:

- **Adaptive shared memory management.** We model shared memory allocation from three dimensions—hardware specifications, model architecture, and dynamic workload—and reduce peak shared memory demand by 50% through K-dimension fine-grained splitting, while enabling cross-operator page reuse to reconstruct efficient pipelines on Ada’s constrained resources.
- **Fine-grained DAG-based automatic search.** We leverage MLIR Lowering [15] to construct PTX-level dependency

DAGs and solidify the optimal execution trace through offline profiling, completely eliminating runtime dynamic decision overhead. Unlike Ansor [31] and other traditional auto-tuning frameworks, Ada-MK’s DAG-level search captures finer-grained parallelism opportunities.

- **Heterogeneous hybrid inference engine.** We embed MegaKernel as a plugin into TensorRT-LLM [21], reusing TensorRT-LLM’s native operators for Prefill and switching to the MegaKernel engine for Decode, achieving both high throughput and low latency—the first industrial deployment of MegaKernel.

Experiments show that Ada-MK reduces end-to-end latency by 10%–50% over vLLM [13], SGLang [33], and vanilla TensorRT-LLM [21] across Qwen model series, and has been deployed in production within Baidu’s commercial online advertising system. The remainder of this paper is organized as follows: §2 presents background and related work; §3 describes the Ada-MK overall architecture; §4 details the three core optimizations; §5 presents experimental evaluation; §6 concludes with future directions.

2 Background and Related Work

2.1 LLM GPU Kernel Techniques: From Local to Global Fusion

GPU operator optimization has evolved from local to global fusion, forming several distinct research directions:

Traditional Kernel Fusion merges adjacent kernels to reduce launch counts, but still relies on global memory for intermediate data exchange, making it difficult to circumvent memory bandwidth and data dependency bottlenecks at extreme model scales. CUDAGraph reduces launch overhead by capturing and replaying kernel sequences, but cannot achieve deep inter-operator fusion.

Compiler-level operator fusion. Triton [25] achieves compile-time operator fusion through block-level programming abstractions, while Hidet [6] provides finer-grained scheduling control via a task-mapping paradigm. These approaches improve fusion depth but still operate on a “kernel-per-operator” execution model, unable to eliminate operator switching overhead. Additionally, MCFuser [29] achieves high-performance fusion for memory-bound compute-intensive operator chains, Neptune [30] enables advanced fusion of reduction operators by breaking loop dependencies, and Mag-neto [4] focuses on coordinated optimization of parallel operator structures.

Single-operator extreme fusion. FlashAttention [3] fuses the multiple steps of attention into a single kernel through IO-aware tiling, achieving breakthrough memory efficiency; FlashAttention-2 [2] further optimizes parallelism and work partitioning. However, these approaches are confined to individual operators.

MegaKernel global fusion [9] integrates an entire computation block into a single persistent kernel, partitions warps within an SM into producer–consumer roles via Warp Specialization, and achieves pipeline parallelism through multi-level storage and asynchronous copies, thoroughly eliminating operator switching overhead and breaking through global memory access limitations. MegaKernel represents the furthest advance in the large-kernel direction, achieving full-chain fusion and persistent computation.

2.2 Analysis of Mainstream Approaches

Stanford’s MegaKernel is built on Tiny-Llama and FlashAttention [3] primitives. It decomposes SM resources into five heterogeneous roles—Loader (asynchronous prefetching), Consumer (tensor computation), Storer (asynchronous writeback), Controller (instruction dispatch coordination), and Launcher (resource release management)—leveraging asynchronous cooperation and semaphore synchronization for deep pipeline decoupling of computation, memory access, and instruction dispatch. Its implementation draws heavily from NVIDIA CUTLASS [20] Warp-Specialized GEMM designs. The framework focuses on Hopper/Blackwell hardware and hard-codes tile sizes for specific models (e.g., Llama-1B), lacking support for Qwen variants. The codebase contains substantial architecture-specific assembly optimizations, making porting to Ada extremely costly. The initial version supports only low-load Decoder phases, lacking support for long-sequence and large-batch Prefill, leaving a significant gap for industrial deployment.

Mirage MPK [1] is a representative recent work in operator compilation, proposing a tGraph-based multi-level pipeline abstraction that automates the search for tiling strategies and storage allocation. Mirage [27] further enables cross-level optimization through μ Graph unification, discovering novel optimizations combining algebraic transformations, scheduling transformations, and custom kernel generation. OLLIE [32] extends the search space of tensor algebraic expressions through derivation-based transformations, Korch [11] achieves optimal kernel orchestration via operator fusion and constraint optimization, PET [26] discovers optimization opportunities invisible to traditional methods through partially equivalent transformations, and TASO [12] automates computation graph substitution generation and verification. These works advance tensor program optimization from different angles, but none addresses MegaKernel-level global operator fusion. MPK’s Managed Pointer mechanism dynamically determines execution paths at runtime, and the resulting if-else branching degrades instruction issue efficiency, preventing it from reaching hardware peak performance. Its automated search also struggles to converge to global optima within reasonable time for irregular tiles or complex operator chains.

3 Ada-MK Overall Architecture

As illustrated in Figure 1, the Ada-MK system architecture comprises two tightly coupled phases: *Offline MegaKernel Synthesis* (Phase I) and *Online Warp-Specialized Execution* (Phase II).

Phase I: Offline MegaKernel Synthesis. This phase is responsible for computation graph generation and hardware-constrained automatic search. The system first parses the logical structure of the Transformer Decoder and LM Head into a fine-grained DAG. A hardware resource filter then evaluates constraints such as shared memory capacity and register limits, directly pruning invalid branches that would exceed resource budgets. Within the resulting feasible search space, the system performs profiling to identify the optimal DAG execution trace for the target GPU, and serializes this trace for runtime invocation.

Phase II: Online Warp-Specialized Execution. This phase embeds the serialized MegaKernel as a plugin into the TensorRT-LLM inference framework, enabling seamless switching between

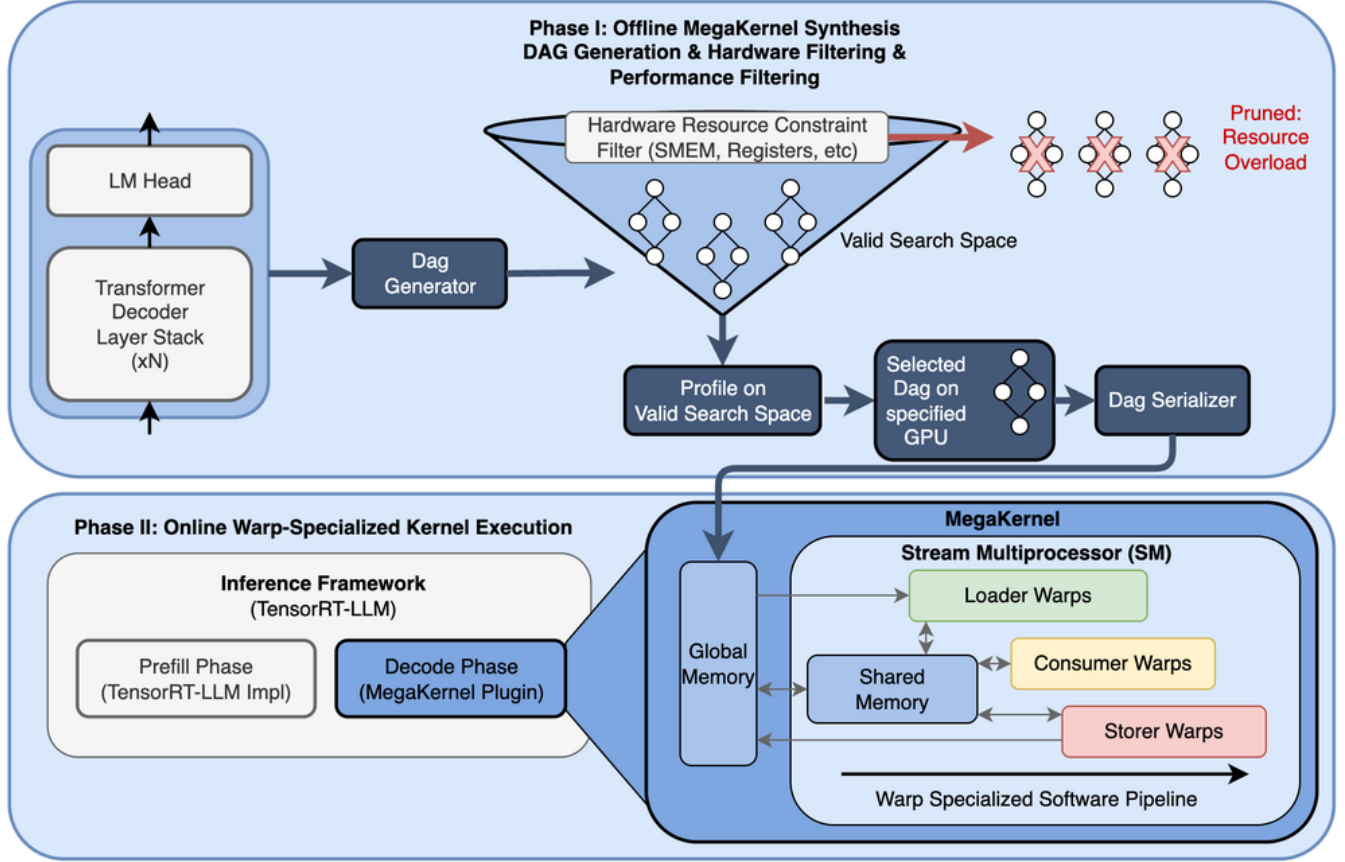


Figure 1: Ada-MK overall architecture. Phase I (Offline MegaKernel Synthesis): the Transformer Decoder and LM Head are parsed into a fine-grained DAG, pruned by hardware resource constraints, and profiled to select the optimal execution trace, which is then serialized. Phase II (Online Warp-Specialized Execution): the serialized MegaKernel is embedded as a TensorRT-LLM plugin, with warp-specialized roles (Loader, Consumer, Storer, Controller, Launcher) collaborating through shared memory to form an efficient software pipeline.

Prefill (using TensorRT-LLM’s native operators) and Decode (using the MegaKernel engine). Within each streaming multiprocessor (SM), MegaKernel constructs an efficient *warp-specialized software pipeline*: the computational resources are spatially partitioned into multiple warp groups—Loader, Consumer, Storer, Controller, and Launcher—that share SM resources and collaborate through shared memory to maximally overlap memory-access latency with computation.

4 Core Optimizations

4.1 Adaptive Shared Memory Management under Resource Constraints

4.1.1 Multi-Dimensional Parameter-Aware Resource Modeling. We model shared memory allocation from three dimensions—hardware specifications, model architecture, and dynamic workload—enabling fine-grained shared memory management:

Hardware-specification awareness. From the GPU hardware specifications [18], we first obtain the maximum available shared

memory per SM. We then partition shared memory into instruction pipeline buffers, synchronization metadata, and computation intermediate buffers, and derive the maximum number of available pages:

$$N_{\text{page}} = \frac{\text{SMem}_{\text{max}} - N_{\text{stage}} \times (\text{Instr}_{\text{buf}} + \text{Semaphores} + \text{Scratch})}{\text{Size}_{\text{page}}} \quad (1)$$

Model-architecture and dynamic-workload awareness. For different model architectures, we compute the shared memory occupied by weights, quantization scales, and other shared data, and compute activation occupancy as a function of batch size. The pipeline depth (N_{stage}) is adaptively adjusted: on resource-constrained devices we enforce capacity constraints, while on resource-rich devices we increase pipeline depth to maximize throughput:

$$N_{\text{stage}} = \frac{N_{\text{page, total}} - N_{\text{page, weight}} - N_{\text{page, scale}} - N_{\text{page, act}}}{N_{\text{page, per stage}}} \quad (2)$$

K-dimension fine-grained splitting. Inspired by a segmented-computation approach, we halve the K -dimension tile. Each loop iteration loads only the weight sub-tile required by the current sub-block, reducing peak shared memory demand by 50%.

4.1.2 Shared Memory Page Reuse. The original Stanford MegaKernel implements page reuse across operators (e.g., releasing a page from operator A for use by operator B). We further optimize this reuse strategy. At execution time, activation pages and weight/output pages are reused based on automated analysis, as detailed in §4.2.

Activation-weight page reuse. Once activations have been loaded from shared memory into registers, the occupied shared memory can be reclaimed and dynamically reallocated to the Loader role for weight storage. This increases the number of pipeline stages (pipeline depth) between the Loader and Consumer, effectively hiding memory-access latency while improving instruction-level parallelism.

Activation-output page reuse. Computed output data typically requires additional shared memory buffering before being written back to global memory. Since MMA computation depends on registers, the shared memory occupied by activations can be released after copying to registers and reallocated for MMA result storage.

4.2 Fine-Grained DAG-Based Automatic Search

4.2.1 Foundational Definitions and Modeling. To precisely characterize MegaKernel’s internal execution logic, we define a formal framework based on directed acyclic graphs (DAGs). Nodes are classified into two categories: data migration (Global→Shared→Register transfers at each level) and computation execution (MMA, dequantization, epilogue), aligned to PTX instruction granularity. Edges capture two types of constraints: *data dependencies* encoding producer-consumer relationships (a successor can execute only after its predecessor completes), and *resource dependencies* encoding competition for limited shared memory pages (a subsequent load must stall when insufficient pages are available).

4.2.2 Automated Pipeline Modeling and Search-Path Optimization. We propose an automated search framework that bridges from low-level IR to pipeline configuration, addressing the limitation that the original MegaKernel lacks fine-grained DAG definitions for efficient pipeline construction.

Step 1: MLIR fine-grained decomposition. To overcome the limitation that operator-level dependency representations obscure parallelism opportunities, we leverage Torch-MLIR’s multi-level transformation architecture [15]. Through deep analysis of MLIR dialects and layer-by-layer Lowering [7], and inspired by Relax’s composable compilation approach for dynamic machine learning [14], we decompose coarse-grained operator black boxes into PTX-level fine-grained dependency traces, providing precise topological support for MegaKernel’s static instruction scheduling and resource management, achieving extreme overlap of computation and I/O.

Step 2: Constructing logical dependencies. Based on MLIR’s alias analysis capabilities, we automatically identify read-after-write

Table 1: MLIR high-level to low-level primitive mapping.

MLIR High	MLIR Low	Physical Semantics	HW Res.
memref.copy	nvgpu.device_async_copy	Global→Shared: async prefetch	HBM, SMem
memref.load	nvgpu.ldmatrix	Shared→Reg: warp-level load	SMem, Regs
quant.dcast	custom.dequant	Dequant: INT4→FP16	ALU, Regs
linalg.matmul	nvgpu.mma.sync	Tensor Core: 16×8×16 MMA	TC
linalg.generic	arith.addf	Epilogue: bias/act fusion	ALU, Regs
memref.copy	vector.transfer_write	Reg→HBM: async writeback	HBM, Regs

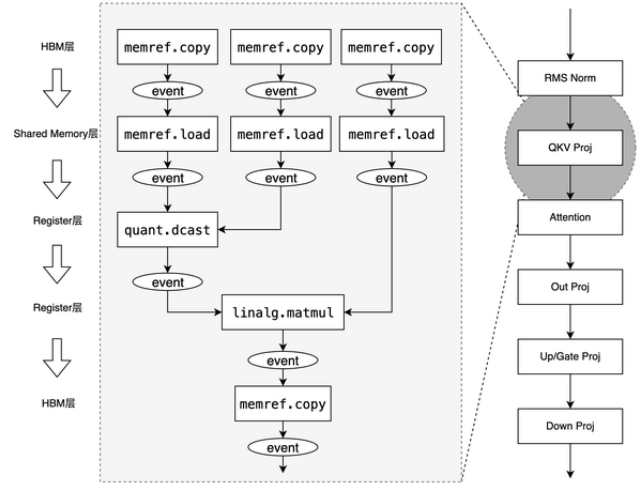


Figure 2: Fine-grained dependency DAG construction from MLIR alias analysis. Write operations identify producers; alias analysis at Read operations determines memory overlap to build RAW edges.

(RAW) dependencies in the instruction sequence, constructing fine-grained dependency DAGs from producers to consumers (Figure 2). Using MLIR’s AliasAnalysis API combined with a lastWriters mapping table, we traverse the instruction sequence, identify memory producers through Write operations, and determine memory overlap at Read operations through alias analysis, thereby automatically constructing RAW dependency DAGs.

Step 3: Schedule and resource candidate-set generation. Pipeline role splitting (DAG-to-Role Mapping). The system assigns DAG nodes to four roles (Figure 3)—Launcher (metadata and page scheduling), Loader (asynchronous weight prefetching), Consumer (input loading, dequantization, and MMA computation), and Storer (result writeback synchronization)—and explores mapping schemes of atomic nodes across different roles. Core strategies include:

- **Load balancing:** Avoid stacking ALU-intensive tasks in a single role; migrate lightweight I/O or preprocessing logic to the Loader role to maximize hardware issue slot utilization.
- **Tiling parameter space:** Simultaneously traverse multiple Block/Warp tile partitioning ratios to seek the theoretical peak compute throughput.

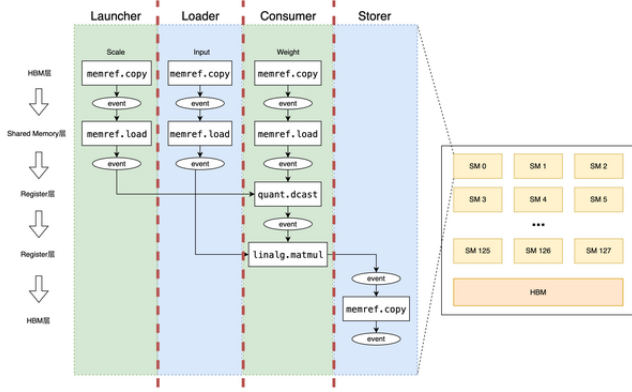


Figure 3: DAG node assignment to four pipeline roles with load balancing and tiling parameter exploration.

Shared-memory-constrained scheduling strategy. Shared memory allocation directly determines whether the pipeline suffers from structural stalls. We define three page states—Empty, Locked, and Ready—identify state conflicts between Loader and Consumer, quantify structural stalls, and compute pipeline duty cycles. The candidate set is pruned through:

- *Early asynchronous prefetching:* Increase the prefetch stride ($N+1 \rightarrow N+2$), trading shared memory space for complete overlap of weight transfer and computation. TileLink [34] demonstrates that tile-centric primitives can effectively overlap compute and communication in distributed settings; Ada-MK applies a similar overlapping principle within a single SM’s software pipeline.
- *Gap filling:* Insert non-critical-path instructions into issue slots left idle by synchronization primitives.
- *Address permutation:* Remap logical pages to physical address offsets, reducing bank conflicts through interleaved reuse. Hexcute [28] provides constraint-solving references for address permutation, and Axe [10] provides a unified layout abstraction for cross-level memory mapping.
- *Role rebalancing:* Forward lightweight tasks (e.g., dequantization) to idle Loader roles, accelerating page turnover.

Step 4: Static trace search and execution-path solidification. Heuristic offline search. Within the space defined by logical dependencies and physical constraints, we simulate multiple page arrangements and role mappings through offline profiling, locking in the execution trace with the highest theoretical duty cycle. Unlike Ansor [31], which employs hierarchical search spaces with evolutionary search, and Pruner [22], which adopts a Draft-then-Verify search acceleration mechanism, Ada-MK’s DAG-level search operates directly on PTX-level dependency graphs, capturing finer-grained parallelism opportunities and effectively handling complex irregular dependencies that traditional MegaKernels cannot address.

Execution-path solidification. Frameworks such as MPK [1] introduce runtime if-else branches that cause instruction issue delays and pipeline bubbles. We employ *path solidification* to serialize the optimal execution path directly into kernel code. This paradigm

shift from “online adaptation” to “offline planning” eliminates the need for runtime resource-state probing, removing the instruction-width overhead of dynamic decision-making.

In summary, this section achieves exhaustive search and locking of physically optimal execution traces in the offline phase through MLIR fine-grained decomposition and DAG dependency modeling, comprehensively considering compute-unit load and storage-page constraints. This “offline search, online reuse” strategy trades compile-time search cost for runtime determinism.

4.2.3 Overall Effectiveness. We systematically analyze the combined performance gains from the fine-grained DAG automatic search mechanism. Experimental results show that our approach achieves approximately 30% performance improvement in the Decode phase compared to the original Stanford MegaKernel, with gains arising from two dimensions.

Systematic elimination of spurious dependencies. The original MegaKernel implementation contains implicit serialization barriers introduced by static compilation strategies that hinder full exploitation of instruction-level parallelism. Through fine-grained DAG dependency tracing under data-flow correlation (RAW/WAR) constraints, we achieve two classes of decoupling optimizations:

Asynchronous prefetching and logical decoupling. DAG analysis confirms no data-flow dependency between RMS Norm weights and QKV computation instructions, enabling their load timing to be advanced to the physical page release point, hiding HBM access latency through asynchronous transfer. Similarly, V-Cache and K-Cache (historical portion) loads in the attention operator are advanced into the pipeline window overlapping with preceding computation, removing historical KV-Cache transfer from the critical path (Figure 4).

Pseudo-dependency elimination and streaming reduction. In SwiGLU and similar dual-path gated activation structures, conventional implementations trigger Reduce only after both Up/Gate GEMM paths complete, forming a coarse-grained synchronization barrier. Through fine-grained DAG modeling, we bind the Reduce start condition to the readiness of the corresponding input component, enabling Up-Reduce to proceed in parallel during Gate-GEMM execution, achieving cross-path streaming reduction.

Auto-tuned optimal pipeline configuration. Static configuration of warp role allocation and shared memory page scheduling cannot accommodate throughput differences across operators. Through offline profiling-driven auto-tuning, we achieve configuration convergence in two dimensions:

Warp allocation refinement. Reducing Consumer warp count from 16 to 8 (Figure 5) decreases single-GEMM output scale while aligning Storer’s cross-block Reduce latency with Consumer’s computation latency, effectively mitigating throughput mismatch among pipeline roles. Additionally, extending the pipeline stage count from 2 to 4 further buffers bubbles caused by execution-latency differences across roles, suppressing pipeline stalls.

Page in-place reuse optimization. Auto-tuning combined with circular page allocation dynamically identifies shared memory page slots eligible for early release, enabling time-multiplexed reuse of weight and activation pages under WAR constraints, effectively

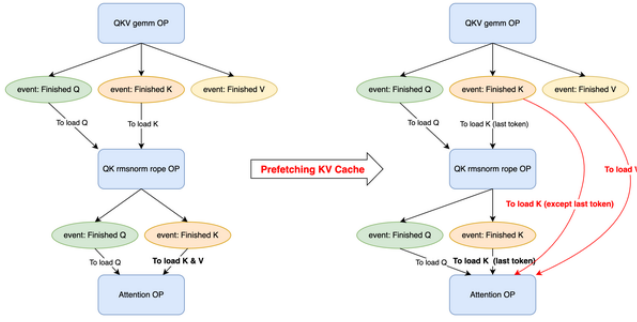


Figure 4: Asynchronous prefetching decouples RMS Norm loads and KV-Cache loads from QKV computation.

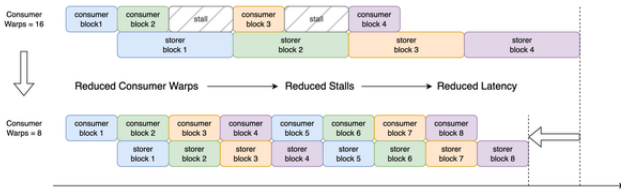


Figure 5: Warp allocation refinement: Consumer warps reduced from 16 to 8 with pipeline stages extended from 2 to 4, aligning role latencies and reducing pipeline stalls.

improving the utilization efficiency of the limited shared memory space.

Combining both optimization dimensions, our approach achieves systematic improvement in MegaKernel pipeline efficiency on Ada’s resource-constrained architecture through the synergy of dependency decoupling and auto-tuning.

4.3 Industrial Deployment and End-to-End Optimization

4.3.1 Heterogeneous Hybrid Inference Engine with Integrated MegaKernel. MegaKernel demonstrates significant advantages in the Decode phase, but lags behind mature TensorRT-LLM in large-scale parallel computation efficiency during the Prefill phase [21]. Three deep-rooted reasons explain this: (1) **Decode is IO-bound:** single-token generation involves minimal computation, with the bottleneck in memory access (weight + KV Cache loading); MegaKernel’s inter-operator parallelism deeply overlaps computation and I/O for significant gains. (2) **Prefill is compute-bound:** long-sequence large-batch computation saturates GPU compute units, yielding diminishing returns from MegaKernel’s IO-compute overlap. (3) **Kernel launch overhead differential:** Decode launches thousands of kernels per token (overhead $\sim 14.6\%$), while Prefill executes only one forward pass where launch overhead is negligible, preventing MegaKernel’s core advantage from being fully exploited.

Moreover, production environments have accumulated extensive business capabilities built on TensorRT-LLM (e.g., prefix-tree constrained decoding, integrated generation-discrimination). Completely replacing the TensorRT-LLM engine would incur prohibitive engineering migration costs. We therefore embed MegaKernel

deeply into TensorRT-LLM’s execution topology via a Plugin mechanism, constructing a heterogeneous hybrid inference engine that combines the strengths of both:

- **Hierarchical operator replacement:** We preserve TensorRT-LLM’s mature implementations for peripheral logic (Embedding, KV Cache management) and replace only the core Transformer Block with a custom MegaKernel plugin, minimizing the modification scope.
- **Phase-adaptive execution:** The Prefill phase retains TensorRT-LLM’s native fused operators, leveraging their high parallelism for large-scale token scenarios; the Decode phase automatically switches to the MegaKernel engine, eliminating launch overhead and memory-access bottlenecks through full-chain kernel fusion.
- **Zero-cost business capability reuse:** Business logic such as prefix-tree constrained decoding and integrated generation-discrimination requires no secondary development for MegaKernel, directly reusing TensorRT-LLM’s existing implementations. This approach simultaneously accommodates Prefill’s high throughput and Decode’s low latency without additional business refactoring costs, providing a viable path for MegaKernel’s large-scale industrial deployment.

4.3.2 MegaKernel Quantization Implementation. Building on adaptive shared memory management, we further incorporate quantization optimizations [17] into MegaKernel. This serves two purposes: quantization compresses weight volume, alleviating Ada’s shared memory capacity bottleneck and enabling more pipeline stages; and Ada’s inference workloads remain memory-bandwidth-bound, so quantization directly reduces memory pressure and improves throughput. The implementation comprises three components:

TensorCore-aware weight reorder. Inspired by QServe [17], we perform offline weight reordering so that each thread can efficiently access weights during inference. At runtime, the Consumer role directly loads quantized weights already aligned to TensorCore access patterns via the LDMATRIX instruction, eliminating the need to write dequantized weights back to shared memory and avoiding runtime layout transformation overhead. Tilus [5] provides reference for layout optimization of arbitrary bit-width low-precision data.

K-dimension multi-level pipeline computation. Combining K-dimension splitting with quantization, the shared memory required per iteration is compressed from 64 KB to 32 KB, reducing page occupancy from 4 to 2. The freed shared memory capacity is used to extend the pipeline stage count, further deepening compute-memory overlap.

Small-batch Padding I/O optimization. Under small batch sizes, Tensor Core instructions require fixed tile dimensions, causing significant padding redundancy when effective data is insufficient. We employ vector-level fine-grained loading with explicit register reorganization to bypass traditional tile-level ldmatrix mechanisms, eliminating redundant I/O of invalid padding data between HBM and registers. This optimization draws on Tilus [5]’s automatic vectorization and instruction selection strategies, as well as AWQ [16]’s activation-aware weight quantization acceleration.

5 Experimental Evaluation

5.1 Experimental Objectives and Research Questions

We systematically evaluate the performance gains of MegaKernel optimizations within TensorRT-LLM [21] and compare against mainstream open-source inference frameworks vLLM [13], SGLang [33], and vanilla TensorRT-LLM [21]. Experiments focus on throughput across different input/output lengths, batch sizes, and task workloads, addressing the following research questions:

- (1) Can MegaKernel significantly improve TensorRT-LLM throughput in low-latency, small-batch inference scenarios?
- (2) Does MegaKernel’s benefit persist when sequence lengths extend from fixed short sequences to real task datasets?
- (3) Does Ada-MK retain advantages over high-throughput frameworks such as vLLM and SGLang at larger batch sizes?
- (4) Is MegaKernel’s performance benefit consistent across model versions?

5.2 Experimental Setup

5.2.1 Compared Frameworks. All frameworks are evaluated in offline batch mode: a fixed batch of requests is submitted simultaneously, and throughput is computed after collecting all generation results. This mode precisely controls concurrency, eliminating interference from online scheduler differences on throughput measurements, and is suitable for horizontal comparison centered on operator execution efficiency.

5.2.2 Model and Quantization Configuration. We cover two Qwen-series models of similar scale but different versions to validate the generality of optimization benefits. Note that the two models have different parameter counts (1.7B vs. 1.5B); cross-model comparisons aim to verify MegaKernel’s generality under identical quantization, and do not exclude the influence of parameter-count differences on absolute performance.

5.2.3 Workload Configuration. Experiments include two workload types:

- (1) **Fixed short-sequence workload:** Input length 64 tokens, output length 12 tokens (denoted in64/out12). This scenario evaluates framework throughput under low-latency, short-output generation tasks.
- (2) **Real-dataset workload:** CSL and Human-eval datasets, with context lengths primarily distributed in the ~ 200 –1000 token range, evaluating performance stability under medium-to-long input scenarios.

All experiments are conducted across batch sizes of 1, 2, 4, 8, and 16. The evaluation metric is generation throughput in tokens/s; higher values indicate better inference performance.

5.2.4 Experimental Environment. All experiments are conducted on a single server with an Intel Xeon Platinum 8558 processor (96 cores / 192 threads), 1 TB DDR5 memory, and a single NVIDIA L20 GPU [18] (48 GB GDDR6, memory bandwidth 864 GB/s, Compute Capability 8.9). The OS kernel is Linux 5.10.0, GPU driver 535.161.07, CUDA 12.2. Each framework runs in an isolated Docker container. Each experiment exclusively occupied a single GPU, eliminating multi-task resource contention.

Table 2: Compared inference frameworks.

Framework	Version	Description
vLLM [13]	v0.19.0	High-throughput LLM inference with efficient KV Cache management
SGLang [33]	v0.5.10	Structured generation and high-performance serving
TRT-LLM [21]	v1.1.0rc5	NVIDIA native inference (baseline)
Ada-MK	—	TRT-LLM + MegaKernel (ours)

Table 3: Model configurations.

Model	Parameters	Quantization
Qwen3-1.7B [24]	1.7B	GPTQ-W4A16 [8]
Qwen2.5-1.5B [23]	1.5B	GPTQ-W4A16 [8]

5.3 Fixed Short-Sequence Experiments

Under fixed input length 64 and output length 12 (simulating short-input short-output business scenarios), the throughput results across frameworks for Qwen3-1.7B and Qwen2.5-1.5B are shown in Figure 6.

The results demonstrate that MegaKernel delivers significant benefits in short-sequence output scenarios. Compared to vanilla TensorRT-LLM, Ada-MK maintains $>10\%$ improvement across all batch sizes, with a 23.6% improvement at BS=1. This indicates that MegaKernel not only improves large-batch throughput but also significantly reduces execution overhead in small-batch scenarios. On Qwen2.5-1.5B, Ada-MK similarly achieves the highest throughput across all batch sizes. Although the improvement over vanilla TensorRT-LLM is slightly lower than on Qwen3-1.7B, it maintains a stable gain of 6.7%–15.6%, confirming that MegaKernel’s optimization is not limited to a single model version.

5.4 Real-Dataset Experiments

5.4.1 CSL Dataset. The CSL dataset corresponds to medium-length context inputs, reflecting inference workloads closer to real tasks. The throughput results for Qwen3-1.7B GPTQ-W4A16 on the CSL dataset are shown in Figure 7.

In the CSL scenario, Ada-MK achieves the best throughput from BS=1 to BS=8, confirming that MegaKernel effectively improves TensorRT-LLM’s execution efficiency under medium-to-long input scenarios. However, at BS=16, vLLM achieves the highest throughput, with Ada-MK lagging vLLM by 3.5% and essentially matching SGLang. This phenomenon indicates that as batch size and sequence length increase, vLLM/SGLang’s advantages in request scheduling, KV Cache management, or parallel scaling begin to emerge.

5.4.2 Human-eval Dataset. The Human-eval dataset further evaluates performance in code-generation tasks. The results for Qwen3-1.7B GPTQ-W4A16 on this dataset are shown in Figure 8.

Unlike the CSL scenario, Ada-MK maintains the highest throughput on the Human-eval dataset even at BS=16. Compared to vLLM and SGLang, its advantage narrows to 1.6% and 3.5% respectively

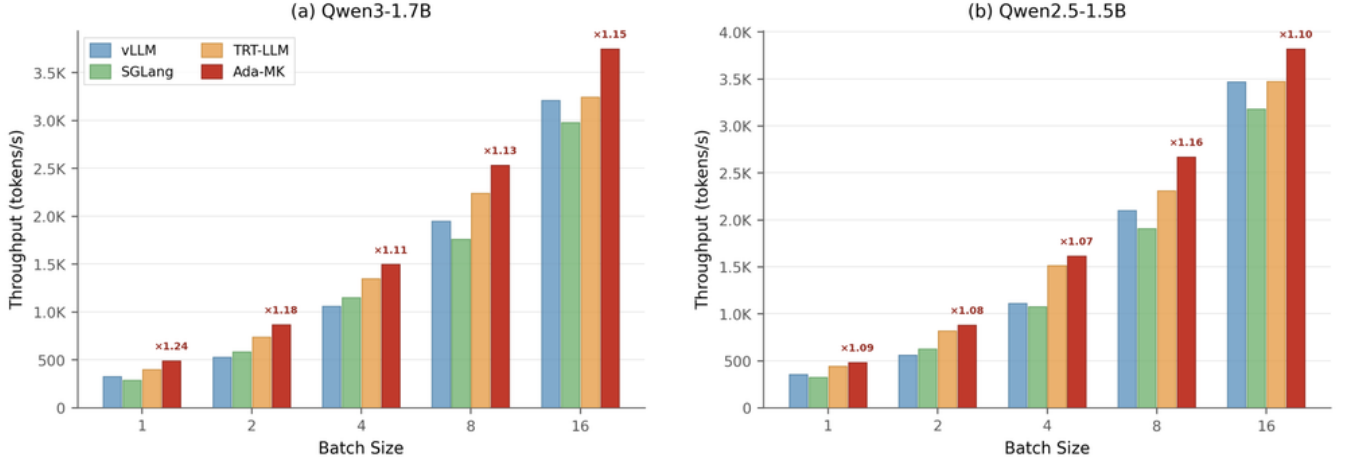


Figure 6: End-to-end throughput comparison on fixed short sequences (input=64, output=12) for (a) Qwen3-1.7B and (b) Qwen2.5-1.5B, both quantized with GPTQ-W4A16. Four inference frameworks are compared: vLLM, SGLang, vanilla TensorRT-LLM, and Ada-MK (ours). Numbers above the Ada-MK bars indicate speedup over vanilla TensorRT-LLM. Ada-MK achieves the highest throughput across all batch sizes, with the most significant gains at small batch sizes (up to 23.6% for Qwen3-1.7B at BS=1).

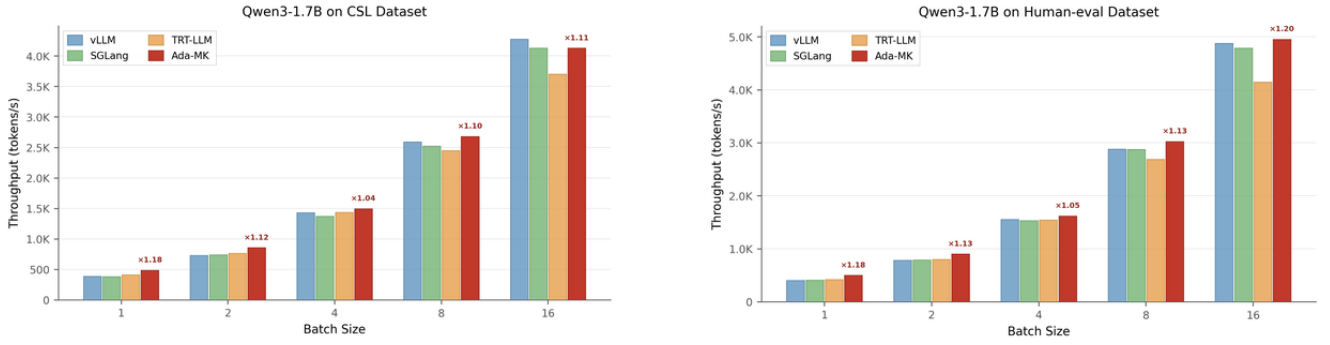


Figure 7: End-to-end throughput comparison on the CSL dataset (Qwen3-1.7B GPTQ-W4A16). Ada-MK achieves the highest throughput at BS=1 through BS=8, while vLLM surpasses Ada-MK at BS=16 by 3.5%, indicating that system-level scheduling advantages emerge under high-concurrency long-context scenarios.

Figure 8: End-to-end throughput comparison on the Human-eval dataset (Qwen3-1.7B GPTQ-W4A16). Ada-MK maintains the highest throughput even at BS=16, with a 19.5% improvement over vanilla TensorRT-LLM, demonstrating strong scalability on code-generation workloads.

at large batch sizes, but it still achieves a significant 19.5% improvement over vanilla TensorRT-LLM. This indicates that MegaKernel provides stable improvement to TensorRT-LLM’s kernel execution efficiency and maintains good scalability on code-generation workloads.

5.5 Comprehensive Analysis and Conclusions

5.5.1 Stable Gains over TensorRT-LLM. Across all experimental results, Ada-MK achieves positive improvement over vanilla TensorRT-LLM in all tested scenarios and batch sizes. The improvement reaches up to 23.6% in fixed short-sequence scenarios and maintains 4.0%–19.5% in real-dataset scenarios, demonstrating that MegaKernel’s optimization is not specific to a single input pattern or model

configuration, but provides universal acceleration for TensorRT-LLM’s execution path.

5.5.2 Most Significant Gains in Small-Batch, Short-Sequence Scenarios. In BS=1/2 low-batch scenarios, Ada-MK’s advantage is most pronounced. For Qwen3-1.7B’s in64/out12 scenario, improvements reach 50.2% and 64.5% over vLLM and 71.9% and 49.3% over SGLang. This confirms that MegaKernel effectively reduces kernel scheduling and execution overhead in small-batch inference, making it particularly suitable for low-latency online inference, interactive requests, and short-text generation tasks.

5.5.3 Narrowing Advantage in Large-Batch, Long-Sequence Scenarios. As batch size increases and input sequences lengthen, Ada-MK’s advantage over vLLM/SGLang gradually narrows. In the CSL dataset at BS=16, vLLM achieves the highest throughput, with

Ada-MK lagging by 3.5%. This indicates that in high-concurrency, long-context scenarios, operator/kernel-level fusion optimization alone may be insufficient to fully compensate for differences arising from system-level scheduling and memory management.

5.5.4 Cross-Model Consistency. In fixed short-sequence scenarios, both Qwen3-1.7B and Qwen2.5-1.5B show Ada-MK achieving the highest throughput across all batch sizes. This result demonstrates that MegaKernel’s optimization benefits exhibit cross-model consistency and can generalize to different model versions of similar scale and identical quantization.

Overall, MegaKernel significantly improves TensorRT-LLM’s inference throughput on GPTQ-W4A16 quantized models, with the most prominent gains in short-sequence, small-batch scenarios. Compared to vLLM and SGLang, Ada-MK achieves comprehensive leadership in fixed short-sequence scenarios and maintains stable advantages at low-to-medium batch sizes in CSL and Human-eval real-dataset scenarios. In high-batch-size, medium-to-long sequence scenarios, Ada-MK’s advantage narrows, with vLLM/SGLang demonstrating better system-level scalability in some cases. MegaKernel is an effective optimization for improving TensorRT-LLM inference performance, particularly suited for low-latency, small-batch, short-output online inference tasks.

6 Conclusion

This paper addresses the deployment challenges of MegaKernel on resource-constrained NVIDIA Ada architectures and proposes the Ada-MK optimization framework. Through multi-dimensional adaptive shared memory management, we reduce peak shared memory demand by 50% and reconstruct efficient pipelines on constrained hardware. Based on MLIR fine-grained DAG decomposition and offline trace search, we solidify execution paths at compile time, eliminating runtime dynamic decision overhead. Through a heterogeneous hybrid engine that embeds MegaKernel as a TensorRT-LLM plugin, we simultaneously accommodate Prefill’s high throughput and Decode’s low latency without additional business refactoring costs. Experimental results demonstrate that Ada-MK achieves significant end-to-end latency improvements over vLLM [13], SGLang [33], and TensorRT-LLM [21] baselines across multiple batch sizes and task types, representing the first large-scale industrial deployment of MegaKernel in a commercial online advertising system. Future work will explore Ada-MK’s adaptation and migration to larger model scales and next-generation Blackwell architectures.

References

- [1] X. Cheng, Z. Zhang, Y. Zhou, J. Ji, J. Jiang, Z. Zhao, Z. Xiao, Z. Ye, Y. Huang, R. Lai, H. Jin, B. Hou, M. Wu, Y. Dong, A. Yip, S. Wang, W. Yang, X. Miao, T. Chen, and Z. Jia. 2025. Mirage Persistent Kernel: A Compiler and Runtime for Mega-Kernelizing Tensor Programs. *arXiv:2512.22219* (2025).
- [2] T. Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Sharing. In *Proc. ICLR*.
- [3] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Proc. NeurIPS*.
- [4] Z. Di, L. Wang, Z. Ma, E. Shao, J. Zhao, Z. Ren, S. Feng, D. Tao, G. Tan, and N. Sun. 2025. Accelerating Parallel Structures in DNNs via Parallel Fusion and Operator Co-Optimization. *ACM Trans. Archit. Code Optim.* 22 (2025), 1–26.
- [5] Y. Ding, B. Hou, X. Zhang, A. Lin, T. Chen, C. H. Yu, Y. Wang, and G. Pekhimenko. 2026. Tilus: A Tile-Level GPGPU Programming Language for Low-Precision Computation. In *Proc. ASPLOS*.
- [6] Y. Ding, C. H. Yu, B. Zheng, Y. Liu, Y. Wang, and G. Pekhimenko. 2023. Hidet: Task-Mapping Programming Paradigm for Deep Learning Tensor Programs. In *Proc. ASPLOS*.
- [7] S. Feng, B. Hou, H. Jin, W. Lin, J. Shao, R. Lai, Z. Ye, L. Zheng, C. H. Yu, Y. Yu, and T. Chen. 2023. TensorIR: An Abstraction for Automatic Tensorized Program Optimization. In *Proc. ASPLOS*.
- [8] E. Frantar, S. Ashkboos, T. Hoeftler, and D. Alistarh. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In *Proc. ICLR*.
- [9] HazyResearch. 2025. Look Ma, No Bubbles! Designing a Low-Latency Megakernel for Llama-1B. <https://github.com/HazyResearch/Megakernels>. (2025).
- [10] B. Hou, H. Jin, G. Wang, J. Chen, Y. Cai, L. Yang, Z. Ye, Y. Ding, R. Lai, and T. Chen. 2026. Axe: A Simple Unified Layout Abstraction for Machine Learning Compilers. *arXiv:2601.19092* (2026).
- [11] M. Hu, A. Venkatram, S. Biswas, B. Marimuthu, B. Hou, G. Oliaro, H. Wang, L. Zheng, X. Miao, J. Zhai, and Z. Jia. 2024. Korch: Optimal Kernel Orchestration for Tensor Programs. In *Proc. ASPLOS*.
- [12] Z. Jia, O. Padon, J. J. Thomas, T. Warsawski, M. Zaharia, and A. Aiken. 2019. TASO: Optimizing Deep Learning Computation with Automatic Generation of Graph Substitutions. In *Proc. SOSR*.
- [13] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proc. SOSR*.
- [14] R. Lai, J. Shao, S. Feng, S. Lyubomirsky, B. Hou, W. Lin, Z. Ye, H. Jin, Y. Jin, J. Liu, L. Jin, Y. Cai, Z. Jiang, Y. Wu, S. Park, P. Srivastava, J. Roesch, T. Mowry, and T. Chen. 2025. Relax: Composable Abstractions for End-to-End Dynamic Machine Learning. In *Proc. ASPLOS*.
- [15] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko. 2021. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *Proc. CGO*.
- [16] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han. 2024. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. In *Proc. MLSys*.
- [17] Y. Lin, H. Tang, S. Yang, et al. 2025. QServe: W4A8KV4 Quantization and System Co-Design for Efficient LLM Serving. In *Proc. MLSys*, Vol. 7.
- [18] NVIDIA. 2022. NVIDIA Ada Lovelace GPU Architecture Whitepaper.
- [19] NVIDIA. 2022. NVIDIA H100 Tensor Core GPU Architecture Whitepaper.
- [20] NVIDIA. 2023. CUTLASS: CUDA Templates for Linear Algebra Subroutines and Solvers. <https://github.com/NVIDIA/cutlass>.
- [21] NVIDIA. 2024. TensorRT-LLM: High-Performance LLM Inference. <https://github.com/NVIDIA/TensorRT-LLM>.
- [22] L. Qiao, J. Shi, X. Hao, X. Fang, S. Zhang, M. Zhao, Z. Zhu, J. Chen, H. An, X. Tang, B. Li, H. Yuan, and X. Wang. 2025. Pruner: A Draft-then-Verify Exploration Mechanism to Accelerate Tensor Program Tuning. In *Proc. ASPLOS*.
- [23] Qwen Team. 2024. Qwen2.5 Technical Report. *arXiv:2412.15115* (2024).
- [24] Qwen Team. 2025. Qwen3 Technical Report. *arXiv:2505.09388* (2025).
- [25] P. Tillet, H. T. Kung, and D. Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *Proc. MAPL*.
- [26] H. Wang, J. Zhai, M. Gao, F. Zhang, T. Wang, Z. Ma, S. Tang, L. Zheng, W. Wang, K. Rong, Y. Chen, and Z. Jia. 2023. PET: Optimizing Tensor Programs with Partially Equivalent Transformations and Automated Corrections. *IEEE Trans. Comput.* 72 (2023), 3546–3560.
- [27] M. Wu, X. Cheng, S. Liu, C. Shi, J. Ji, M. Ao, P. Velliengiri, X. Miao, O. Padon, and Z. Jia. 2024. Mirage: A Multi-Level Superoptimizer for Tensor Programs. In *Proc. PLDI*.
- [28] X. Zhang, Y. Ding, B. Sun, Y. Hu, T. Shpeisman, and G. Pekhimenko. 2026. Hexcute: A Compiler Framework for Automating Layout Synthesis in GPU Programs. In *Proc. CGO*.
- [29] Z. Zhang, D. Yang, X. Zhou, and D. Cheng. 2024. MCFuser: High-Performance and Rapid Fusion of Memory-Bound Compute-Intensive Operators. In *Proc. SC*.
- [30] Y. Zhao, E. Johnson, P. Chatarasi, V. S. Adve, and S. Misailovic. 2025. Neptune: Advanced ML Operator Fusion for Locality and Parallelism on GPUs. *arXiv:2510.08726* (2025).
- [31] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen, J. E. Gonzalez, and I. Stoica. 2020. Ansor: Generating High-Performance Tensor Programs for Deep Learning. In *Proc. OSDI*.
- [32] L. Zheng, H. Wang, J. Zhai, M. Hu, Z. Ma, T. Wang, S. Tang, L. Xie, K. Huang, and Z. Jia. 2022. OLLIE: Derivation-based Tensor Program Optimizer. *arXiv:2208.02025* (2022).
- [33] L. Zheng, L. Yin, Z. Xie, J. Sun, C. Cui, E. Xie, and H. Zhang. 2025. SGLang: Efficient Execution of Structured Language Model Programs. In *Proc. ICLR*.
- [34] S. Zheng, J. Fang, X. Zheng, Q. Hou, W. Bao, N. Zheng, Z. Jiang, D. Wang, J. Ye, H. Lin, L.-W. Chang, and X. Liu. 2025. TileLink: Generating Efficient Compute-Communication Overlapping Kernels using Tile-Centric Primitives. *arXiv:2503.20313* (2025).